

## DL Assignment 1 — Boston Housing Price Prediction using Deep Learning

This assignment performs:

### House Price Prediction using Deep Neural Network

using:

- TensorFlow/Keras
- regression
- Boston Housing dataset

## 1. What We Have To Do in This Assignment?

We have to:

1. load Boston Housing dataset
2. preprocess data
3. build deep learning regression model
4. train model
5. predict house prices
6. evaluate prediction performance

## Main Goal

To predict:

### house prices

based on features like:

- crime rate
- number of rooms
- tax rate
- highway access
- etc.

## 2. What Type of Problem is This?

This is:

### Regression Problem

because output is:

continuous numerical value

Example:

House price = 25.6

NOT categories/classes.

## 3. Overall Workflow

Load Dataset



Feature Scaling



Build Neural Network



Compile Model  
↓  
Train Model  
↓  
Predict Prices  
↓  
Evaluate Performance

## CELL 1 — Install Libraries

```
!pip install --user tensorflow scikit-learn pandas numpy matplotlib seaborn
```

### Purpose

Installs required libraries.

### Libraries Used

| Library      | Purpose                 |
|--------------|-------------------------|
| TensorFlow   | deep learning           |
| scikit-learn | preprocessing & metrics |
| pandas       | dataframe               |
| numpy        | arrays                  |
| matplotlib   | plotting                |
| seaborn      | graphs                  |

## CELL 2 — Import Libraries

```
import tensorflow as tf
from sklearn.preprocessing import StandardScaler
from sklearn import metrics
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")
```

### Line-by-Line

#### TensorFlow

```
import tensorflow as tf
Used for:
```

**building neural network**

#### StandardScaler

```
from sklearn.preprocessing import StandardScaler
Used for:
```

**feature scaling**

## metrics

from sklearn import metrics

Used for:

- MAE
- MSE
- R<sup>2</sup> score

## pandas

import pandas as pd

Used for:

## graph plotting

## numpy

import numpy as np

Numerical operations.

## matplotlib

import matplotlib.pyplot as plt

Graph plotting.

## seaborn

import seaborn as sns

Regression plot visualization.

## warnings

warnings.filterwarnings("ignore")

Hides warning messages.

## CELL 3 — Load Dataset

```
# Load Boston Housing Dataset
```

```
boston = tf.keras.datasets.boston_housing
```

```
(x_train, y_train), (x_test, y_test) = boston.load_data()
```

```
print(x_train.shape, y_train.shape)
```

```
print(x_test.shape, y_test.shape)
```

## What is Boston Housing Dataset?

Dataset containing:

- house features
- house prices

## Features

Total:

13 features

Examples:

- crime rate
- rooms
- tax
- pollution

# Output

House price.

## Train-Test Split

### Dataset Purpose

x\_train training features

y\_train training prices

x\_test testing features

y\_test testing prices

## Shape Output

```
print(x_train.shape)
```

Example:

(404, 13)

Meaning:

- 404 houses
- 13 features each

## CELL 4 — Feature Scaling

# Feature Scaling

```
scaler = StandardScaler()
```

```
x_train_scaled = scaler.fit_transform(x_train)
```

```
x_test_scaled = scaler.transform(x_test)
```

```
y_train_scaled = scaler.fit_transform(y_train.reshape(-1, 1))
```

```
y_test_scaled = scaler.transform(y_test.reshape(-1, 1))
```

## Why Scaling Needed?

Different features have different ranges.

Example:

| Feature | Range |
|---------|-------|
|---------|-------|

|       |      |
|-------|------|
| Rooms | 1–10 |
|-------|------|

|            |        |
|------------|--------|
| Crime Rate | 0–1000 |
|------------|--------|

Large values dominate learning.

Scaling normalizes data.

## Standardization Formula

$$z = \frac{x - \mu}{\sigma}$$

## fit\_transform()

```
scaler.fit_transform(x_train)
```

Does:

1. calculate mean/std
2. transform data

## transform()

```
scaler.transform(x_test)
```

Uses training statistics only.

Test data should NOT influence training.

## reshape(-1,1)

```
y_train.reshape(-1,1)
```

Converts:

1D → 2D array

Required by StandardScaler.

## CELL 5 — Build Deep Learning Model

```
model = tf.keras.models.Sequential([
    tf.keras.layers.Input(shape=(13,), name='input-layer'),
    tf.keras.layers.Dense(100, name='hidden-layer-2'),
    tf.keras.layers.BatchNormalization(name='hidden-layer-3'),
    tf.keras.layers.Dense(50, name='hidden-layer-4'),
    tf.keras.layers.Dense(1, name='output-layer')
])
```

## Model Architecture

13 Inputs

↓

Dense(100)

↓

BatchNorm

↓

Dense(50)

↓

Output(1)

## Input Layer

```
Input(shape=(13,))
```

13 input features.

## Dense(100)

```
Dense(100)
```

Fully connected layer with:

100 neurons

## Fully Connected Means

Every neuron connected to:

**all previous neurons**

## BatchNormalization

```
BatchNormalization()
```

Purpose:

- stabilize learning
- faster training
- reduce internal covariate shift

## Dense(50)

Second hidden layer with:  
50 neurons  
Learns deeper patterns.

## Output Layer

Dense(1)  
One neuron because:

**regression output is single value**

## CELL 6 — Model Summary

`model.summary()`

## Purpose

Displays:

- layers
- output shapes
- parameters

## Total Params

All model parameters.

## Trainable Params

Parameters updated during learning.

## CELL 7 — Compile Model

```
model.compile(  
    optimizer='adam',  
    loss='mse',  
    metrics=['mae']  
)
```

## optimizer='adam'

Controls:

## weight updates

Adam is fast and efficient.

## loss='mse'

Regression loss function.

Formula:

$$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

## metrics=['mae']

Evaluation metric.

Formula:

$$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

## CELL 8 — Train Model

```
history = model.fit(
    x_train_scaled,
    y_train_scaled,
    batch_size=32,
    epochs=20,
    validation_data=(x_test_scaled, y_test_scaled)
)
```

## What Happens Here?

Model:

- predicts
- calculates error
- updates weights
- learns patterns

## batch\_size=32

Processes:

32 samples at a time

## epochs=20

Complete dataset processed:

20 times

## validation\_data

Used to evaluate:

**model performance during training**

## history Variable

Stores:

- loss
- MAE
- validation metrics

for each epoch.

## CELL 9 — Plot Metrics Graph

```
pd.DataFrame(history.history).plot(figsize=(10, 7))
plt.title("Metrics graph")
plt.show()
```

## Purpose

Plots:

- training loss
- validation loss
- MAE

## Good Graph

loss ↓

MAE ↓

Model improving.

## CELL 10 — Prediction

```
y_pred_scaled = model.predict(x_test_scaled)
# Convert back to original scale
y_pred = scaler.inverse_transform(y_pred_scaled)
```

### **model.predict()**

Predicts scaled house prices.

### **inverse\_transform()**

Converts scaled predictions back to:

**original house price values**

## CELL 11 — Regression Plot

```
sns.regplot(x=y_test, y=y_pred)
plt.title("Regression Line for Predicted values")
plt.show()
```

### **Purpose**

Compares:

- actual prices
- predicted prices

### **Good Regression Plot**

Points close to line mean:

**accurate predictions**

## CELL 12 — Evaluation Metrics

```
def regression_metrics_display(y_test, y_pred):
    print("MAE is", metrics.mean_absolute_error(y_test, y_pred))
    print("MSE is", metrics.mean_squared_error(y_test, y_pred))
    print("R2 score is", metrics.r2_score(y_test, y_pred))
    regression_metrics_display(y_test, y_pred)
```

### **MAE**

Average prediction error.

Lower is better.

### **MSE**

Squared prediction error.

Penalizes large mistakes.

### **R<sup>2</sup> Score**

Measures model quality.

Range:



| Value | Meaning |
|-------|---------|
| 1     | perfect |
| 0     | poor    |
| <0    | bad     |

## Good $R^2$

Closer to:

1

## 4 Important Concepts Used

| Concept             | Purpose                  |
|---------------------|--------------------------|
| Regression          | predict continuous value |
| Feature Scaling     | normalize data           |
| Deep Neural Network | learn patterns           |
| Batch Normalization | stabilize training       |

## Important Viva Questions

### Q1. What type of problem is Boston Housing?

Regression problem.

### Q2. Why scaling needed?

To normalize feature ranges.

### Q3. Why MSE used?

Because this is regression.

### Q4. Why output layer has 1 neuron?

Single numerical output.

### Q5. What is BatchNormalization?

Technique to stabilize and speed training.

### Q6. Difference between MAE and MSE?

MAE uses absolute errors, MSE uses squared errors.

### Q7. What does $R^2$ score indicate?

How well model explains variance in data.

## Short Practical Explanation

This program predicts Boston house prices using a deep neural network in TensorFlow. The dataset is preprocessed using feature scaling, trained using Dense and BatchNormalization layers, and evaluated using MAE, MSE, and  $R^2$  score. Regression plots are used to visualize prediction performance.